

Day 6 - Algorithms & Pathfinding

Enemies that find their way around a wall, a search that stays fast as the world grows: this is where data structures meet gameplay. Today you sharpen the fundamentals (search, sort, Big-O instinct) and then build the one every game studio asks about, **A* pathfinding**. This is the heaviest day of the week, so read about the Red Blob A* read early.

Learning objectives

By the end of today you can:

- [ ] Implement binary search and state its precondition and complexity.
- [ ] Explain the sorting basics and reach for `std::sort` correctly.
- [ ] Reason about Big-O well enough to compare two approaches.
- [ ] Represent a graph (grid / adjacency) suitable for search.
- [ ] Implement A* with an admissible heuristic and trace why it finds a shortest path.

Concepts

Binary search

Halve a **sorted** window each step: a million elements take at most ~20 comparisons, O(log n) against O(n) for a scan. Track the window as a half-open interval `[lo, hi)`: `lo` is included, `hi` is excluded, so the whole vector is `[0, size)` and `lo == hi` means the window is empty and the target absent. In real code reach for `std::lower_bound` / `std::binary_search`.

```
// returns index of target, or -1; requires v sorted ascending
int binarySearch(const std::vector<int>& v, int target){
    int lo = 0, hi = static_cast<int>(v.size());   // window = whole vector, [lo, hi)
    while (lo < hi) {                              // loop while window non-empty
        int mid = lo + (hi - lo) / 2;              // not (lo+hi)/2, since that sum can overflow int
        // three cases: v[mid] hits, target is right of mid, target is left of mid.
        // You write these (ex00): which bound moves, and does mid stay in the
        // window? Remember hi is exclusive: keep mid on the wrong side and
        // the window never shrinks, so the loop runs forever.
    }
```

```
    return -1;                                  // window emptied
without a hit
}
```

The "which half?" step is only valid because the data is sorted: everything left of `mid` is ≤ `v[mid]`, everything right is ≥, so one comparison discards half the window. Trace on `{1, 3, 5, 7, 9}` (ex00's sorted vector), target `7`:

1. `[0,5)` → `mid=2`, `v[2]=5 < 7` → `lo=3`
2. `[3,5)` → `mid=4`, `v[4]=9 > 7` → `hi=4`
3. `[3,4)` → `mid=3`, `v[3]=7` → return `3` ✓

❌ **Common Mistake:** running binary search on unsorted data. The "discard half the window" step is only valid on sorted input, so it returns `-1` for elements that are sitting right there.

✅ **Good Practice:** sort once up front (or keep the container sorted) and treat "is it sorted?" as the function's stated precondition.

Sorting basics

You'll almost never write a sort by hand: `std::sort` is O(n log n), while the naive ones (bubble/insertion) are O(n^2) and only win on tiny or nearly-sorted input. What you *do* write is the **comparator**, a function `std::sort` calls on pairs of elements to ask one question: "should `a` come before `b`?" Return `true` to put `a` first.

```
struct Entity { std::string name; int hp; };
std::vector<Entity> entities = { {"orc", 7}, {"slime", 2}, {"drake", 12} };

std::sort(entities.begin(), entities.end(),          // sorts the range
[begin, end)
          [](const Entity& a, const Entity& b) { return a.hp < b.hp; });
// lower hp = "comes before": slime(2) orc(7) drake(12). Flip to a.hp > b.hp
for descending.
```

Without a comparator, `std::sort` uses `<`, which is fine for `int`. An `Entity` has no built-in `<`, so you must say what "before" means. **Stable** means equal elements keep their original relative order: if two entities both had 7 hp, `std::stable_sort` would keep them in arrival order, while `std::sort` is free to swap them.

❌ **Common Mistake:** a comparator that isn't a strict-weak ordering, e.g. returning `<=`, which claims two equal elements each come before the other. That's undefined behaviour in `std::sort`: crashes or infinite loops on some inputs.

✅ **Good Practice:** compare with strict `<` (or `>`), so equal elements return `false` in both directions.

Big-O awareness

Not formal proofs, just the instinct to say "that's $O(n^2)$, the grid is 1000×1000, so no." Know the common classes and roughly where each stops being usable.

Big-O	Name	~n it handles interactively	Example
O(1)	constant	any	hash lookup, array index
O(log n)	logarithmic	any	binary search
O(n)	linear	millions	single scan
O(n log n)	linearithmic	millions	`std::sort`
$O(n^2)$	quadratic	a few thousand	nested loops, naive pair check
$O(2^n)$/O(n!)	exponential/factorial	~20 / ~10	brute-force subsets/permutations

❌ **Common Mistake:** optimising a constant factor when the *class* is the problem. An $O(n^2)$ inner loop won't be saved by micro-tuning.

✅ **Good Practice:** work out the complexity class first and change the algorithm or the data structure; tune constants only once the class is right.

Graphs

A graph is just "things plus which things connect": **nodes** and **edges**. A game map is already one, with each open cell a node and an edge to each open cell beside it. You don't build a node/edge structure to prove that: the grid *is* the graph, and one function makes it searchable. That function is `neighbours(cell)`, which answers "from this cell, where can I step?". That's the only question A* will ever ask the map.

```
struct Cell { int x, y; };

std::vector<Cell> neighbours(const Grid& g, Cell c){
    std::vector<Cell> out;
    const int dx[] = {1, -1, 0, 0}, dy[] = {0, 0, 1, -1};   // pair i = one
step: (+1,0) right,
    for (int i = 0; i < 4; ++i) {                           // (-1,0) left,
(0,+1) down, (0,-1) up
        Cell n{c.x + dx[i], c.y + dy[i]};
        if (g.inBounds(n) && g.isOpen(n)) out.push_back(n); // silently
drop off-map cells and walls
    }
    return out;
}
```

On ex01's `simple.txt`, `neighbours({0, 0})` (the `S` corner) returns only `(1,0)` and `(0,1)`, because left and up fall off the map. Because the filtering lives here, whoever calls `neighbours` never thinks about walls or bounds again.

❌ **Common Mistake:** letting the storage order leak into your coordinates. `std::getline` gives you the map as rows, so in the stored lines the row (`y`) comes first, the opposite of the `(x, y)` the map format speaks. Spread that flip across the code and one missed swap finds `G` in the wrong place or walks through walls.

✅ **Good Practice:** bury the flip in a single private helper (`char at(Cell c) const`) and think in `(x, y)` everywhere else.

A* pathfinding

A* explores outward from the start, always expanding the cell that looks cheapest *overall*: `f = g + h`, where `g` is the cost actually paid to reach the cell and `h` is an optimistic estimate of what's left to the goal. `g` alone is Dijkstra, which spreads evenly in every direction. Adding `h` steers the search toward the goal, and stays correct as long as `h` never overestimates (see **Heuristics**).

Two pieces of bookkeeping: the **open set** holds discovered-but-not-yet-expanded cells in a priority queue popped lowest-`f` first, and `cameFrom[n]` records which cell discovered `n`. When the goal is popped, walk `cameFrom` backwards to rebuild the path.

```
// sketch. g[]: best cost so far per cell; open: min-heap keyed on f = g + h
// every cell's g starts at "infinity" (e.g. INT_MAX, or an absent map
entry)
g[start] = 0;                                       // paid nothing yet;
forget this and the search is dead
open.push({start, h(start, goal)});
while (!open.empty()) {
    Node cur = open.top(); open.pop();             // cheapest-looking cell
in the frontier
    if (cur.cell == goal) return reconstruct(cameFrom, goal);   // popped =
its cost is settled
    for (Cell nb : neighbours(grid, cur.cell)) {
        int tentativeG = g[cur.cell] + 1;          // cost to reach nb via
cur (uniform step cost 1)
        if (tentativeG < g[nb]) {                   // strictly cheaper
than any known route to nb
            g[nb] = tentativeG;                     // record the better
route
            cameFrom[nb] = cur.cell;                // and who found it,
for reconstruction
            open.push({nb, tentativeG + h(nb, goal)});   // f = paid so far
+ optimistic remainder
        }
    }
}
return {};   // frontier drained without reaching the goal: no path exists
```

Trace on ex02's `simple.txt`, with `S = (0,0)`, `G = (3,3)`, and Manhattan `h`:

1. pop `S` (`g=0, h=6, f=6`); its neighbours `(1,0)` and `(0,1)` both get `g=1, h=5, f=6`
2. each step along a shortest route trades 1 paid (`g+1`) for 1 less estimated (`h−1`), so those cells all sit at `f=6` and pop before any costlier cell

3. pop `G` at `g=6`, then walk `cameFrom` back to `S`: the length-6 path ex02 prints

❌ **Common Mistake:** a plain `std::queue` instead of a priority queue. Pops then ignore `f`, so the search spreads evenly like BFS instead of steering toward the goal. On a uniform grid it still finds a shortest path, just slowly (it expands nearly every cell); with varying step costs it returns wrong paths.

✅ **Good Practice:** pop from a min-heap keyed on `f = g + h` (`std::priority_queue` with a comparator), so the cheapest-looking cell is always expanded first.

Heuristics

For A* to return a shortest path, `h` must be **admissible**: it must never *overestimate* the true remaining cost. On a 4-directional grid, **Manhattan distance** (`|dx| + |dy|`) is admissible and tight; **Euclidean** is admissible but looser (underestimates because you can't move diagonally). An inadmissible `h` is faster but can return a non-optimal path.

```
int manhattan(Cell a, Cell b){ return std::abs(a.x - b.x) + std::abs(a.y -
b.y); }
```

❌ **Common Mistake:** an `h` that overestimates, e.g. Manhattan on a grid that allows diagonal moves. A* settles a cell too early and returns a path that isn't shortest.

✅ **Good Practice:** pick a heuristic that can never exceed the true remaining cost: Manhattan for 4-directional movement, Euclidean once diagonals are allowed.

Exercises

Map format

Exercises ex01 and ex02 read an ASCII grid from a file or stdin: `.` = open, `#` = wall, `S` = start, `G` = goal. Movement is 4-directional, uniform step cost. Cells are addressed `(x, y)` with `x` = column, `y` = row, both zero-based from the top-left.

`maps/simple.txt`:

```
S....
.###.
...#.
.#.G.
.....
```

`maps/blocked.txt` (no path):

```
S.#..
.#.#.
.#.#.
```

```
.#..G
###..
```

Expected: `simple.txt` has a shortest path of length 6 (steps); `blocked.txt` reports "no path". Provide at least these two plus one open map with an obstacle to route around.

ex00 - Binary search

.	.	.
💢	**Difficulty**	⭐⭐
🧠	**Objectives**	*Implement `binarySearch` over numbers taken from the command line: sort them with `std::sort` (incl. one descending view via lambda) and report the target's index for any input.*
📁	**Turn in directory**	`ex00/`
❗	**Turn in files**	**main.cpp, Makefile**
🛠️	**Skills required**	`std::sort` *with a comparator, binary search on a half-open interval, parsing numbers from* `argv`*, edge-case testing.*
🔍	**Search keywords**	*C++ std::sort lambda comparator, binary search implementation, lower_bound, O(log n) vs O(n), half-open interval*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<string>`, `<vector>`, `<algorithm>`

- **Goal:** Implement `binarySearch` (declaration below) and a program `./search <target> <numbers...>`: the first argument is the target, the rest are the data. Sort the data ascending with `std::sort` and print it, print one **descending** view via a lambda comparator (see **Sorting basics**), then re-sort ascending and print the search result. Binary name: `search`.
- **Done when:** Any invocation prints the right index (or `-1`): target at the first and last position, target absent, a single number, no numbers at all. Your evaluator will try their own numbers, so don't special-case the runs below.

The descending sort is display only; the search runs on the ascending data (the precondition):

```
// requires v sorted ascending; returns the index of target, or -1
int binarySearch(const std::vector<int>& v, int target);
```

Output example:

```
$ ./search 7 5 3 9 1 7
sorted asc:  1 3 5 7 9
sorted desc: 9 7 5 3 1
find 7 -> index 3
$ ./search 4 5 3 9 1 7
sorted asc:  1 3 5 7 9
sorted desc: 9 7 5 3 1
find 4 -> -1
$ ./search 42
sorted asc:
sorted desc:
find 42 -> -1
```

`binarySearch` only works because the data is sorted ascending. The descending view proves you can re-order for display without breaking the search's precondition.

ex01 - Grid to graph

.	.	.
💢	**Difficulty**	⭐⭐⭐
🧠	**Objectives**	*Parse a map file into a `Grid` class, locate `S` and `G`, and answer `neighbours(cell)` queries from the command line, listing only the open, in-bounds cells around any queried cell.*
📁	**Turn in directory**	`ex01/`
❗	**Turn in files**	**Grid.hpp/.cpp, main.cpp, maps/, Makefile**
🛠️	**Skills required**	*Parsing an ASCII grid into a `vector<string>`, computing 4-neighbours, bounds/wall checks, locating marker cells.*
🔍	**Search keywords**	*C++ parse ascii grid, getline vector, 4-directional neighbours, grid graph, bounds checking*
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<fstream>`, `<string>`, `<vector>`

- **Goal:** Build the `Grid` class below. It loads a map file (see **Map format**) and answers the one question ex02's A* will ask it, `neighbours(cell)`: *from this cell, where can I step?* The program is `./grid <map> <x> <y> [<x> <y> ...]`. It prints, in order: the map itself, one line each for where `S` and `G` are, then one `neighbours(x,y)` line per queried cell, listing neighbours in the fixed scan order right, left, down, up (the direction array in **Graphs**). Hint: `std::getline` line-by-line into a `std::vector<std::string>` is a perfectly good grid; keep the row/column lookup in one private helper so everything else speaks `(x, y)` (see the **Graphs** common mistake). Binary name: `grid`.
- **Done when:** Every `neighbours` line lists exactly the open, in-bounds 4-neighbours of the queried cell, with walls and off-map cells dropped, and `S` / `G` are located correctly on every provided map. Your evaluator will query cells of their own (corners, cells beside walls), so don't hard-code the answers.

```
// Grid.hpp
struct Cell { int x, y; };

class Grid {
public:
    static Grid load(const std::string& path);
    bool inBounds(Cell c) const;
    bool isOpen(Cell c) const;                    // in bounds and not '#'
    std::vector<Cell> neighbours(Cell c) const;  // walkable 4-neighbours
    Cell start() const;   // the 'S'
    Cell goal()  const;   // the 'G'
};
```

Output example:

```
$ ./grid maps/simple.txt 0 0 2 2 4 4
S....
.###.
...#.
.#.G.
.....
start S = (0,0)
goal  G = (3,3)
neighbours(0,0): (1,0) (0,1)
neighbours(2,2): (1,2) (2,3)
neighbours(4,4): (3,4) (4,3)
```

Each query proves the filtering: `(0,0)` drops its two off-map sides, `(2,2)` drops the wall above and the wall to its right, `(4,4)` drops the two off-map sides of the far corner. Because that filtering lives inside `neighbours`, ex02's A* never has to think about walls or bounds.

ex02 - A*

.	.	.
💢	**Difficulty**	⭐⭐⭐⭐⭐
🧠	**Objectives**	*On top of ex01's `Grid`, implement A* with the Manhattan heuristic and a `std::priority_queue` open set; reconstruct and print the shortest path, and report "no path" when the goal is unreachable.*
📁	**Turn in directory**	`ex02/`
❗	**Turn in files**	**Grid.hpp/.cpp (from ex01), main.cpp, maps/, Makefile**
🛠️	**Skills required**	*A* search, `priority_queue` open set keyed on `f = g + h`, Manhattan heuristic, `cameFrom` path reconstruction.*
🔍	**Search keywords**	*C++ A* pathfinding, priority_queue open set, manhattan heuristic admissible, reconstruct path cameFrom, shortest path grid**
🔮	**Command required**	`c++ -Wall -Wextra -Werror -std=c++17`
📚	**Allowed libraries**	`<iostream>`, `<fstream>`, `<string>`, `<vector>`, `<queue>`, `<map>`, `<cmath>`

- **Goal:** On top of ex01's `Grid`, implement A* with the Manhattan heuristic and a `std::priority_queue` open set (see **A* pathfinding**). Reconstruct the shortest path and mark it on the grid with (keep `S` and `G` visible). Report "no path" when the goal is unreachable. Binary name: `astar`.
- **Done when:** Returns a shortest path on every solvable map (length 6 on `simple.txt`, marked as below) and prints "no path" on `blocked.txt`.

The priority queue is what makes this A* and not BFS: it pops the most promising cell first.

```
struct Node { Cell cell; int f; };                          // f = g + h
struct ByF { bool operator()(const Node& a, const Node& b){ return a.f >
b.f; } };
std::priority_queue<Node, std::vector<Node>, ByF> open;   // min-heap on f

// ... A* loop (see Concepts) fills cameFrom ...
std::vector<Cell> path = reconstruct(cameFrom, goal);     // start .. goal
std::cout << "path length: " << path.size() - 1 << '\n';  // edges, not
cells
```

Output example:

```
$ ./astar maps/simple.txt
S....
*###.
***#.
.#*G.
.....
path length: 6

$ ./astar maps/blocked.txt
no path
```

Ordering the open set on `f = g + h` with an admissible Manhattan `h` guarantees the first time you pop `G` you reached it via a shortest path. A plain FIFO queue would give you BFS instead.

Constraints (every exercise)

- Compiles with `c++ -Wall -Wextra -Werror -std=c++17`.
- Use STL containers from Day 4 (e.g. a `std::priority_queue` for the open set).
- State the complexity of what you wrote (in a comment).
- Follows Day 1 conventions; no leaks.

How you're evaluated

An evaluation will be automatically scheduled with a peer and start at 6pm. Submit all the work you have done so far even if its not 100% complete.

Turning in your work

1. Create a new repository on GitHub named `gauntlet-day-06`, and keep it **public**. A private repo is one your evaluator cannot clone, and the evaluation cannot start without the code.
2. Commit today's work there, one directory per exercise (`ex00/`, `ex01/`, ...) exactly as each turn-in table says, `Makefile` included.
3. Push before the evaluation. Whatever sits on the default branch when the evaluation starts is what gets read.
4. When the evaluation starts, give your evaluator the link to the repository.

AI-written code is an immediate disqualification. Look things up and ask an AI what something does, that is a tool. Pasting in code you did not write is not.